
Hypothesis Testing of Movie Ratings Data

Data Analysis Project 1

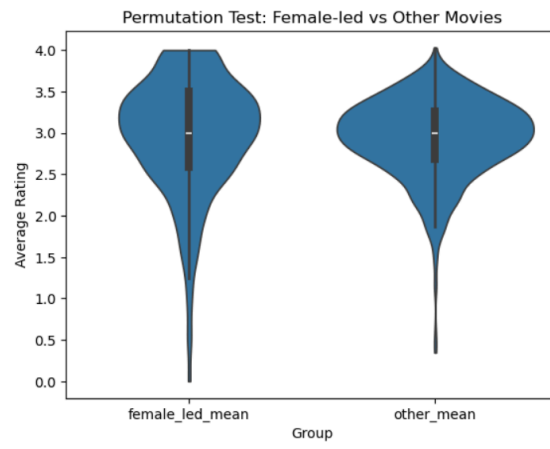
Introduction to Data Science (DS-GA 1001)

11/03/2025

Note: Our aim is to demonstrate the essential skills involved in hypothesis testing. We will be using a real dataset that stems from a replication attempt of published research (Wallisch & Whritner, 2017).

1. Are movies that are more popular (operationalized as having more ratings) rated higher than movies that are less popular?
 - **D(Do):** Movies were split into high and low popularity groups (median = 198 ratings) with mean ratings compared using a **greater Mann-Whitney U test** ($\alpha = 0.005$).
 - **Y(Why):** **MW U Test** was chosen because movie ratings are ordinal and not normal and we are comparing independent groups (high vs low popularity).
 - **F(Find):** Median popularity = 198 ratings with mean rating for high popularity as **2.868** and for low popularity is **2.401**. Parameter $U = 35,404$ with **$p = 1.7e-40$** .
 - **A(Answer):** Since **$p < 0.05$** , the **difference is significant**. Hence, more popular movies are rated higher.
2. Are movies that are newer rated differently than movies that are older?
 - **D(Do):** Movies were split by median year (1999) and compared via **two-sided Mann-Whitney U test**.
 - **Y(Why):** **MW U Test** used to compare group ratings and Spearman's correlation to test year-wise rating trends since both are nonparametric and robust to non-normality.
 - **F(Find):** Median year = 1999 with mean rating for new movies as **2.66** and for old as **2.62**. Parameter $U = 20,949$ with **$p = 0.262$** .
 - **A(Answer):** Since **$p > 0.05$** , there is **no statistically significant difference** between ratings of newer and older movies.
3. Is enjoyment of 'Shrek (2001)' gendered, i.e. do male and female viewers rate it differently?
 - **D(Do):** Shrek (2001) ratings between female and male compared via **two-sided Mann-Whitney U test** after removing missing or invalid gender responses, as ratings were ordinal and non-normal.
 - **Y(Why):** **Test** used is ideal for comparing two independent groups on an ordinal scale.
 - **F(Find):** Mean ratings for females is **3.16** and **3.08** for males. Parameter $U = 96830.5$ with **$p = 0.0505$** .
 - **A(Answer):** Since **$p > 0.05$** , there is **no statistically significant gender difference** in enjoyment of *Shrek*, though female viewers rated it marginally higher on average.
4. What proportion of movies are rated differently by male and female viewers?
 - **D(Do):** Compared male and female viewers' ratings across movies using **two-sided Mann-Whitney U tests** after removing missing values.
 - **Y(Why):** Test used is ideal since ratings are ordinal and not normally distributed, making it more suitable than a t-test for comparing two independent groups.
 - **F(Find):** Across 400 movies, **12.5%** showed significant gender-based rating differences (**$p < 0.005$**).
 - **A(Answer):** **12.5%** of movies show **significant gender-based rating differences**, suggesting gender effects in movies. Although most films are rated similarly, some findings may reflect false positives.
5. Do people who are only children enjoy 'The Lion King (1994)' more than people with siblings?
 - **D(Do):** Compared ratings between only children and those with siblings using a **greater Mann-Whitney U test** after removing missing values.
 - **Y(Why):** The test used is suited for non parametric, ordinal and non-normal data, comparing group ranks.
 - **F(Find):** Only children rated movies slightly lower (**3.348**) than those with siblings (**3.482**). Parameters $U = 52,929.0$ and **$p = 9.7842e-01$** .
 - **A(Answer):** Since **$p > 0.005$** , there is **no significant difference** in enjoyment of *The Lion King (1994)* between only children and those with siblings.
6. What proportion of movies exhibit an "only child effect"?
 - **D(Do):** Ratings for 400 movies were compared between only children and those with siblings using the **two-sided Mann-Whitney U test** on cleaned ordinal data after removing missing values.
 - **Y(Why):** Test used is ideal since ratings are ordinal and not normally distributed, making it more suitable.

- **F(Find):** 1.8% showed a **significant “only child effect”**, indicating that such differences in ratings are rare.
 - **A(Answer):** Since only 1.8% of movies showed significant differences, being an only child appears to have minimal impact on movie enjoyment, with most effects likely due to chance.
7. Do people who like to watch movies socially enjoy ‘The Wolf of Wall Street (2013)’ more than those who prefer to watch them alone?
- **D(Do):** Ratings were compared between social and solo watchers using a **greater Mann-Whitney U test** after excluding missing responses.
 - **Y(Why):** Ratings are ordinal and non-normal, the **Mann-Whitney U test** was used to compare the two independent groups.
 - **F(Find):** Mean for social was 3.03 while 3.14 for solo watchers. The parameters **U = 49,303.5** and **p = 9.4367e-01**.
 - **A(Answer):** Since **p > 0.005**, there is **no significant “social watching”** effect as social and solo watchers rated it similarly, indicating minimal practical difference.
8. What proportion of movies exhibit such a “social watching” effect?
- **D(Do):** Tested whether ratings differed between social and solo watchers using a two-sided **Mann-Whitney U test** on 400 movies, excluding missing values.
 - **Y(Why):** Movie ratings are ordinal and non-normal, the **Mann-Whitney U test** appropriately compares two independent groups.
 - **F(Find):** Only **10 of 400 movies (2.5%)** showed significant social-watching effects.
 - **A(Answer):** Movie enjoyment is largely unaffected by social-versus-solo watching preferences, suggesting minimal or chance-level effects.
9. Is the ratings distribution of ‘Home Alone (1990)’ different from that of ‘Finding Nemo (2003)’?
- **D(Do):** Compared paired ratings using the **Wilcoxon signed-rank test**, including only participants who rated both movies (**n = 810**).
 - **Y(Why):** Since ratings are paired and ordinal, the Wilcoxon test was chosen over a paired t-test to avoid normality assumptions.
 - **F(Find):** Mean rating for Finding Nemo is 3.40 and 3.13 for Home Alone with **W = 41,298.5** and **p = 6.3794e-10**.
 - **A(Answer):** Participants rated *Finding Nemo (2003)* **significantly higher** than *Home Alone (1990)*, confirming a clear preference for the newer film.
10. What proportion of movies exhibit such a “social watching” effect?
- **D(Do):** Compared viewer ratings across movies within eight major film franchises using the **Kruskal-Wallis H test** to assess rating consistency.
 - **Y(Why):** The **Kruskal-Wallis test** was used because ratings are ordinal, not normal and spans multiple independent movie groups within each franchise.
 - **F(Find):** **7 out of 8** franchises showed **significant rating differences** with only *Harry Potter* displaying consistent ratings.
 - **A(Answer):** Most franchises exhibit significant variability in viewer ratings, indicating inconsistent perceived quality across their sequels.
11. **Additional Insight:** We want to check if there is gender bias towards female led movies using permutation tests.
- **D(Do):** Compared mean ratings of female-led vs. other movies using a **Permutation test** (10,000 resamples). List of Female-led movies were filtered using a web browser.
 - **Y(Why):** **Permutation test** to avoid normality and variance assumptions with ordinal, uneven data.
 - **F(Find):** Mean difference ≈ -0.04 and **p = 0.30** shows **no significant difference**.
 - **A(Answer):** Viewers rated female-led and other movies similar overall.



Appendix

```
import pandas as pd
import numpy as np
from scipy.stats import mannwhitneyu, wilcoxon, kruskal, permutation_test
import matplotlib.pyplot as plt
import re
import seaborn as sns
```

```
df = pd.read_csv("movieReplicationSet.csv")
```

```
df.head()
```

The emo tions on the scre en "rub off" on me - for insta nce if some ethin g sad is hap penin g I get	As a mo vie un fold s I star t to have pro ble ms kee pin g trac k of eve nts that hap pen ed ear lier	When wat chin g a mov ie I feel like the thin gs on the scre en are hap pen ing to me	When wat chin g a mo vie I che er or sh out or talk or cur se at the scre en	Black Swa n (2010)	Let the Ri ght One In (2008)	Far go (1996)	Ram bo : Firs t Blo od Pa rt II (1985)	Snatch (2000)	Indiana Jones and the Last Cru sade (1989)	Alien (1979)	Django Uncha ined (2012)	Win g Com man der (1999)	The Life of Davi d Ga le (2003)	Are you an only child? (1: Yes; 0: No; -1: Did not res pond)	Gender iden tity (1 = female; 2 = mal e; 3 = self -descri bed)	When wat chin g mo vie s thi ng s get so int ense tha t I have to sto p wat chin g	Mo vie s cha nge my posi tion on soc ial eco no mic or poli tica l iss ues	When wat chin g a mov ie I get com plet ely imm erse d in the alter nati ve reali ty of the film
														Mo vie s are be st en joy ed alo ne (1: Yes; 0: No; -1: Did not res pond)				

												scar ed									
0	N																				
	a	NaN	4.0	N	3.	N	N	N	N	N	.	1.0	6.0	2.0	5.0	5.0	5.0	1.0	1.0	0	1
	N			N	0	a	a	a	a	a	.										
1	N																				
	a	NaN	1.5	N	Na	N	N	N	N	N	.	3.0	1.0	1.0	6.0	5.0	3.0	2.0	1.0	0	0
	N			N	N	a	a	a	a	a	.										
2	N																				
	a	NaN	Na	N	Na	N	N	N	N	N	.	5.0	4.0	3.0	5.0	5.0	4.0	4.0	1.0	1	0
	N			N	N	a	a	a	a	a	.										
3	N																				
	a	NaN	2.0	N	3.	N	N	N	N	4.	.	3.0	1.0	1.0	4.0	5.0	3.0	1.0	1.0	0	1
	N			N	0	a	a	a	a	0	.										
4	N																				
	a	NaN	3.5	N	0.	N	0.	1.	N	0.	.	2.0	3.0	2.0	5.0	6.0	4.0	4.0	1.0	1	1
	N			N	5	a	5	0	N	0	.										

5 rows × 477 columns

```
df.columns
Index(['The Life of David Gale (2003)', 'Wing Commander (1999)',
      'Django Unchained (2012)', 'Alien (1979)',
      'Indiana Jones and the Last Crusade (1989)', 'Snatch (2000)',
      'Rambo: First Blood Part II (1985)', 'Fargo (1996)',
      'Let the Right One In (2008)', 'Black Swan (2010)',
      ...,
      'When watching a movie I cheer or shout or talk or curse at the screen',
      'When watching a movie I feel like the things on the screen are happening to me',
      'As a movie unfolds I start to have problems keeping track of events that
happened earlier',
      'The emotions on the screen "rub off" on me - for instance if something sad is
happening I get sad or if something frightening is happening I get scared',
      'When watching a movie I get completely immersed in the alternative reality of
the film',
      'Movies change my position on social economic or political issues',
      'When watching movies things get so intense that I have to stop watching',
      'Gender identity (1 = female; 2 = male; 3 = self-described)'])
```

```

    'Are you an only child? (1: Yes; 0: No; -1: Did not respond)',
    'Movies are best enjoyed alone (1: Yes; 0: No; -1: Did not respond)'],
    dtype='object', length=477)
total_rows = len(df)
missing_counts = df.isnull().sum()

```

```
missing_counts
```

```

The Life of David Gale (2003)      1021
Wing Commander (1999)             1026
Django Unchained (2012)           644
Alien (1979)                       808
Indiana Jones and the Last Crusade (1989) 634
...
Movies change my position on social economic or political issues      20
When watching movies things get so intense that I have to stop watching 18
Gender identity (1 = female; 2 = male; 3 = self-described)             24
Are you an only child? (1: Yes; 0: No; -1: Did not respond)            0
Movies are best enjoyed alone (1: Yes; 0: No; -1: Did not respond)      0

```

```
Length: 477, dtype: int64
```

1. Are movies that are more popular (operationalized as having more ratings) rated higher than movies that are less popular? [Hint: You can do a median-split of popularity to determine high vs. low popularity movies]

```

# Extracting only movie ratings
movies = df.iloc[:, :400]

popularity = movies.notna().sum()
mean_rating = movies.mean(skipna=True)

#now we are appending the above two features in the dataset
movie_stats = pd.DataFrame({
    'popularity': popularity,
    'mean_rating': mean_rating
})

```

```

# Median splitting based on popularity

med_popular = movie_stats['popularity'].median()
high_popular = movie_stats[movie_stats['popularity'] > med_popular]
low_popular = movie_stats[movie_stats['popularity'] <= med_popular]

movie_stats.head()

```

	popularit y	mean_ratin g
The Life of David Gale (2003)	76	2.151316

Wing Commander (1999) 71 2.021127

Django Unchained (2012) 453 3.153422

Alien (1979) 289 2.707612

**Indiana Jones and the Last Crusade
(1989)** 463 2.778618

```
# Performing Man-Whitney U test
```

```
U, p = mannwhitneyu(high_popular['mean_rating'], low_popular['mean_rating'],  
alternative='two-sided')
```

```
U , p
```

```
(np.float64(35404.0), np.float64(1.6971433120157929e-40))
```

```
#Computing correlation
```

```
n1, n2 = len(high_popular), len(low_popular)
```

```
ranking = 1 - (2 * U) / (n1 * n2)
```

```
print(f"Median popularity: {med_popular}")
```

```
print(f"Mean rating (high popularity): {high_popular['mean_rating'].mean():.2f}")
```

```
print(f"Mean rating (low popularity): {low_popular['mean_rating'].mean():.2f}")
```

```
print(f"Mann-Whitney U: {U:.1f}, p-value: {p:.2e}")
```

```
print(f"Rank correlation: {ranking:.2f}")
```

```
print(f"n_high = {n1}, n_low = {n2}")
```

```
Median popularity: 197.5
```

```
Mean rating (high popularity): 2.87
```

```
Mean rating (low popularity): 2.40
```

```
Mann-Whitney U: 35404.0, p-value: 1.70e-40
```

```
Rank correlation: -0.77
```

```
n_high = 200, n_low = 200
```

```
#Visualizing data
```

```
plt.figure(figsize=(10,5))
```

```
plt.hist(high_popular['mean_rating'], bins=10, alpha=0.6, label='High popularity',  
density=True)
```

```
plt.hist(low_popular['mean_rating'], bins=10, alpha=0.6, label='Low popularity',  
density=True)
```

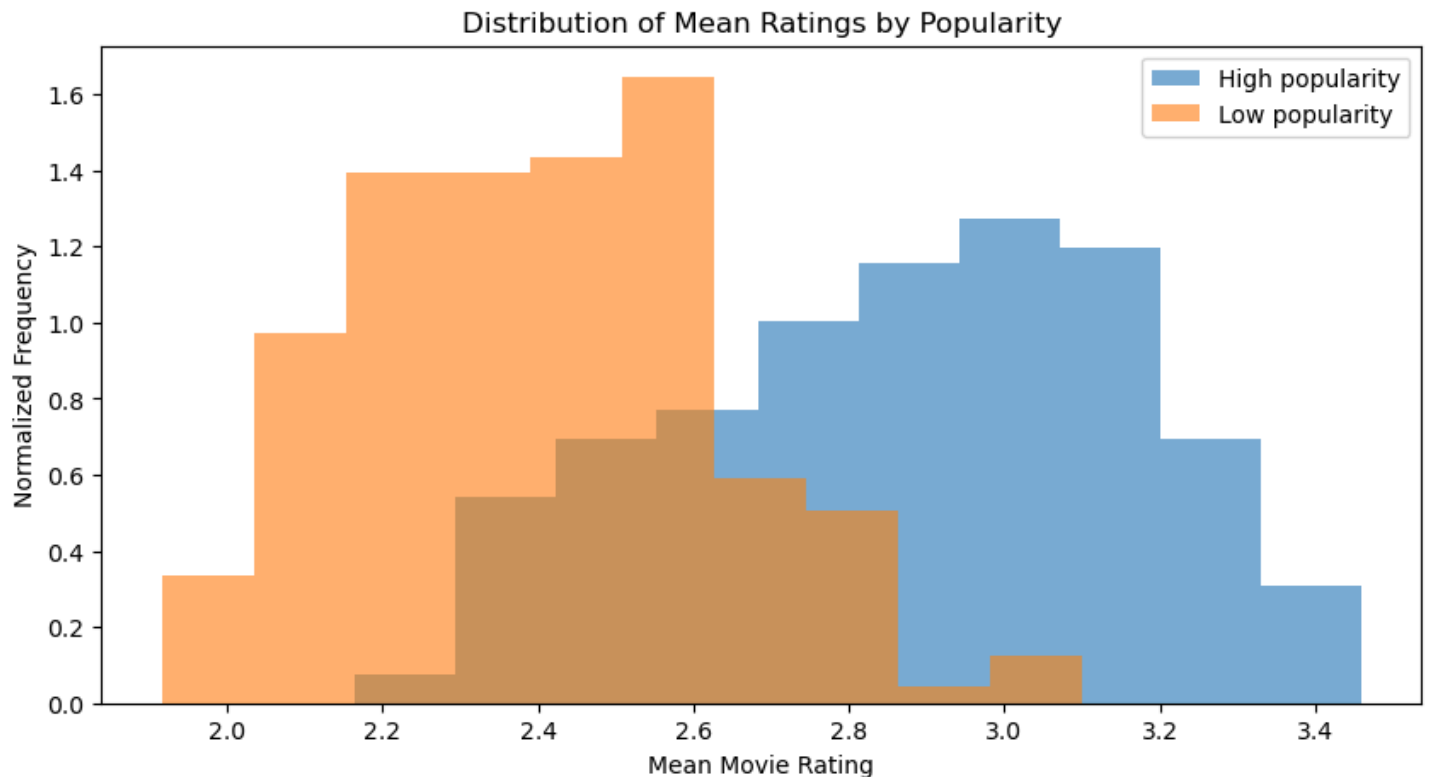
```
plt.xlabel('Mean Movie Rating')
```

```
plt.ylabel('Normalized Frequency')
```

```
plt.title('Distribution of Mean Ratings by Popularity')
```

```
plt.legend()
```

```
plt.show()
```

A (Answer): Given $p < 0.005$ and $r \approx -0.77$, I conclude that less popular movies tend to receive higher ratings than more popular ones. This effect is large and statistically significant. A limitation is that popularity and rating may be correlated through sampling bias (only fans rate niche films), not necessarily film quality.

- Are movies that are newer rated differently than movies that are older? [Hint: Do a median split of year of release to contrast movies in terms of whether they are old or new]

```
# Extracting year from columns using RE function
def extract_year(title):
    match = re.search(r"\((\d{4})\)", title)
    return int(match.group(1)) if match else np.nan

years = [extract_year(c) for c in movies.columns]
mean_ratings = movies.mean(skipna=True)

# Combining into a DataFrame
movie_year = pd.DataFrame({
    'year': years,
    'mean_rating': mean_ratings
}).dropna()

# Median split by year
median_year = movie_year['year'].median()
old_movies = movie_year[movie_year['year'] <= median_year]
new_movies = movie_year[movie_year['year'] > median_year]

[old_movies, new_movies]
```

	year	mean_rating
Wing Commander (1999)	1999	2.021127
Alien (1979)	1979	2.707612
Indiana Jones and the Last Crusade (1989)	1989	2.778618
Rambo: First Blood Part II (1985)	1985	2.365385
Fargo (1996)	1996	2.899606
...	...	...
Patton (1970)	1970	2.278689
Anaconda (1997)	1997	2.295455
Twister (1996)	1996	2.402299
MacArthur (1977)	1977	2.114754
Look Who's Talking (1989)	1989	2.337963

[226 rows x 2 columns],

	year	mean_rating
The Life of David Gale (2003)	2003	2.151316
Django Unchained (2012)	2012	3.153422
Snatch (2000)	2000	2.597656
Let the Right One In (2008)	2008	2.496350
Black Swan (2010)	2010	2.911565
...	...	...
Harry Potter and the Goblet of Fire (2005)	2005	3.299754
Gods and Generals (2003)	2003	2.198413
My Father and My Son (2005)	2005	2.250000
X-Men 2 (2003)	2003	2.914062
Harry Potter and the Chamber of Secrets (2002)	2002	3.272459

[174 rows x 2 columns]]

Mann-Whitney U test

```
U, p = mannwhitneyu(new_movies['mean_rating'], old_movies['mean_rating'],
alternative='two-sided')
```

Rank-biserial correlation

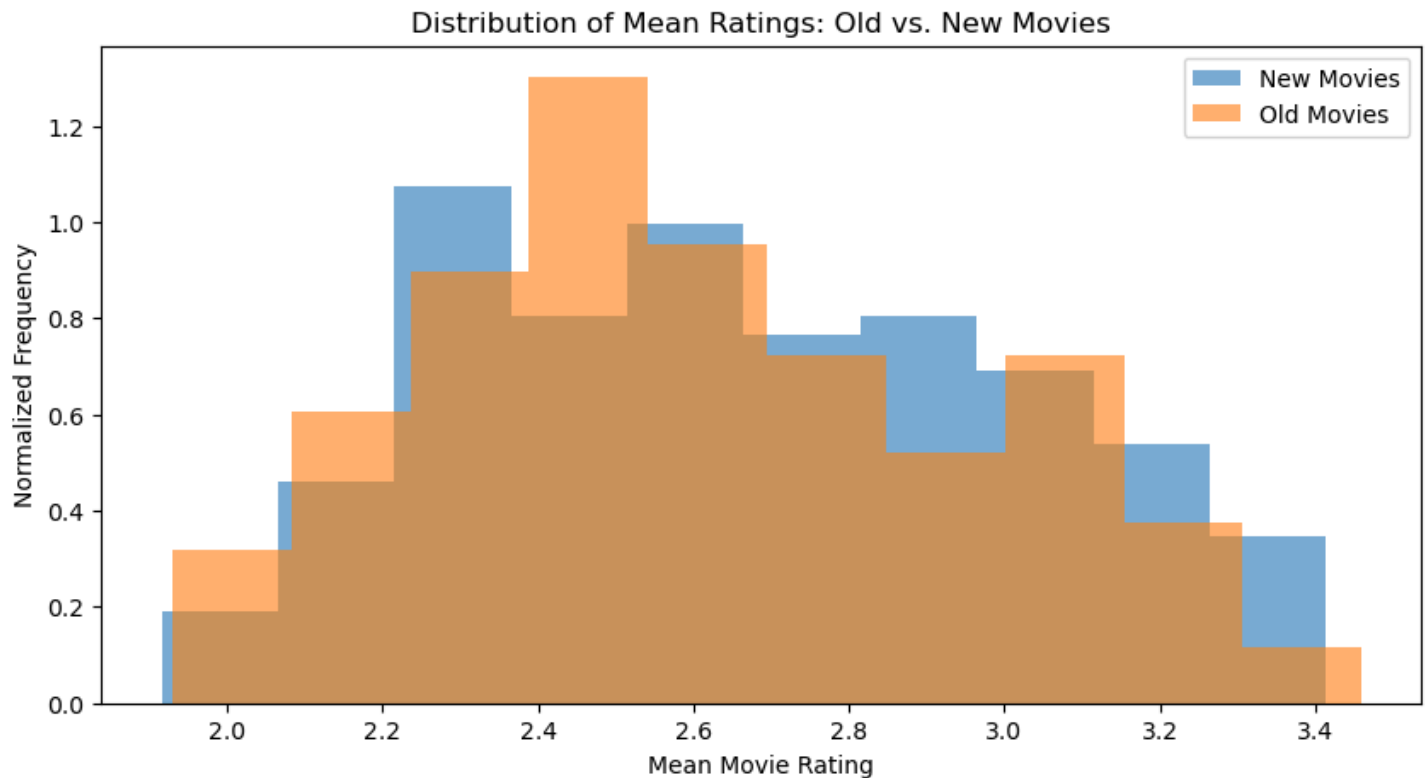
```
n1, n2 = len(new_movies), len(old_movies)
rank_biserial = 1 - (2 * U) / (n1 * n2)
```

```
print(f"Median release year: {median_year}")
print(f"Mean rating (new movies): {new_movies['mean_rating'].mean():.2f}")
print(f"Mean rating (old movies): {old_movies['mean_rating'].mean():.2f}")
print(f"Mann-Whitney U: {U:.1f}, p-value: {p:.2e}")
print(f"Rank-biserial correlation: {rank_biserial:.2f}")
print(f"n_new = {n1}, n_old = {n2}")
```

```
Median release year: 1999.0
Mean rating (new movies): 2.66
Mean rating (old movies): 2.62
Mann-Whitney U: 20949.0, p-value: 2.62e-01
Rank-biserial correlation: -0.07
n_new = 174, n_old = 226
```

```
plt.figure(figsize=(10,5))
```

```
plt.hist(new_movies['mean_rating'], bins=10, alpha=0.6, label='New Movies',
density=True)
plt.hist(old_movies['mean_rating'], bins=10, alpha=0.6, label='Old Movies',
density=True)
plt.xlabel('Mean Movie Rating')
plt.ylabel('Normalized Frequency')
plt.title('Distribution of Mean Ratings: Old vs. New Movies')
plt.legend()
plt.show()
```



```
# Spearman correlation
from scipy.stats import spearmanr

rho, pval = spearmanr(movie_year['year'], movie_year['mean_rating'], nan_policy='omit')

print(f"Spearman correlation (rho): {rho:.3f}")
print(f"p-value: {pval:.3e}")
```

```
Spearman correlation (rho): 0.100
p-value: 4.576e-02
```

3. Is enjoyment of 'Shrek (2001)' gendered, i.e. do male and female viewers rate it differently?

```
# Extracting Shrek ratings and gender
shrek = df['Shrek (2001)']
gender = df.iloc[:, 474]
```

```
# Clean data
```

```

data = pd.DataFrame({'rating': shrek, 'gender': gender})
data = data[(data['gender'].isin([1, 2])) & (data['rating'].notna())]

# Split by gender
female = data[data['gender'] == 1]['rating']
male = data[data['gender'] == 2]['rating']

# Mann-Whitney U Test
u_stat, p_value = mannwhitneyu(female, male, alternative='two-sided')

# Rank-biserial correlation
n1, n2 = len(female), len(male)
r_rb = 1 - (2*u_stat) / (n1*n2)

print(f"U = {u_stat}, p = {p_value:.3e}, r_rb = {r_rb:.3f}")
print(f"Female mean = {female.mean():.2f}, Male mean = {male.mean():.2f}")

U = 96830.5, p = 5.054e-02, r_rb = -0.082
Female mean = 3.16, Male mean = 3.08

```

data

	rating	gender
0	3.0	1.0
1	3.0	1.0
2	3.5	1.0
3	0.5	1.0
4	3.0	1.0
...	...	...
109	3.0	1.0
2		

109 4.0 1.0
3

109 3.5 1.0
4

109 2.5 1.0
5

109 2.0 1.0
6

984 rows × 2 columns

```
data["gender_label"] = data["gender"].replace({1: "Female", 2: "Male"})

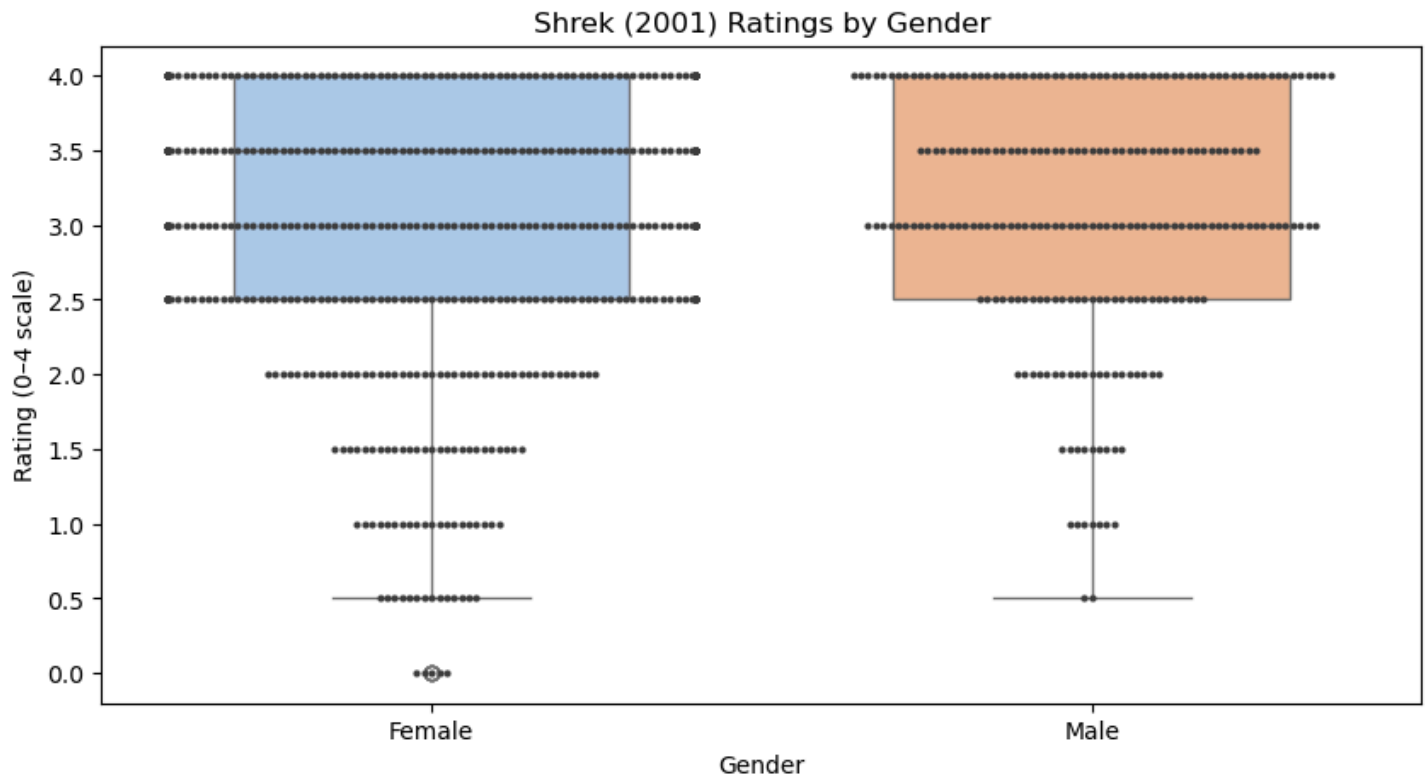
plt.figure(figsize=(10, 5))
sns.boxplot(x="gender_label", y="rating", data=data, palette="pastel", width=0.6)
sns.swarmplot(x="gender_label", y="rating", data=data, color=".25", size=3)

plt.title("Shrek (2001) Ratings by Gender")
plt.xlabel("Gender")
plt.ylabel("Rating (0-4 scale)")
plt.show()
```

C:\Users\adars\AppData\Local\Temp\ipykernel_13112\154758672.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="gender_label", y="rating", data=data, palette="pastel", width=0.6)
C:\Users\adars\anaconda3\Lib\site-packages\seaborn\categorical.py:3399: UserWarning:
41.6% of the points cannot be placed; you may want to decrease the size of the markers
or use stripplot.
warnings.warn(msg, UserWarning)
C:\Users\adars\anaconda3\Lib\site-packages\seaborn\categorical.py:3399: UserWarning:
47.0% of the points cannot be placed; you may want to decrease the size of the markers
or use stripplot.
warnings.warn(msg, UserWarning)
```



4. What proportion of movies are rated differently by male and female viewers?

```
gender = df.iloc[:, 474].replace(-1, pd.NA)

valid_genders = gender.isin([1, 2])
gender = gender[valid_genders]
df_movies = df.loc[valid_genders, df.columns[0:400]]

# --- Initialize counters ---
significant_count = 0
tested_count = 0
alpha = 0.005

# --- Loop over each movie column ---
for col in df_movies.columns:
    ratings = df_movies[col]
    data = pd.DataFrame({"rating": ratings, "gender": gender})
    data = data.dropna()

    if data["gender"].nunique() == 2:
        female = data[data["gender"] == 1]["rating"]
        male = data[data["gender"] == 2]["rating"]

        if len(female) > 0 and len(male) > 0:
            u_stat, p_val = mannwhitneyu(female, male, alternative="two-sided")
            tested_count += 1
            if p_val < alpha:
                significant_count += 1
```

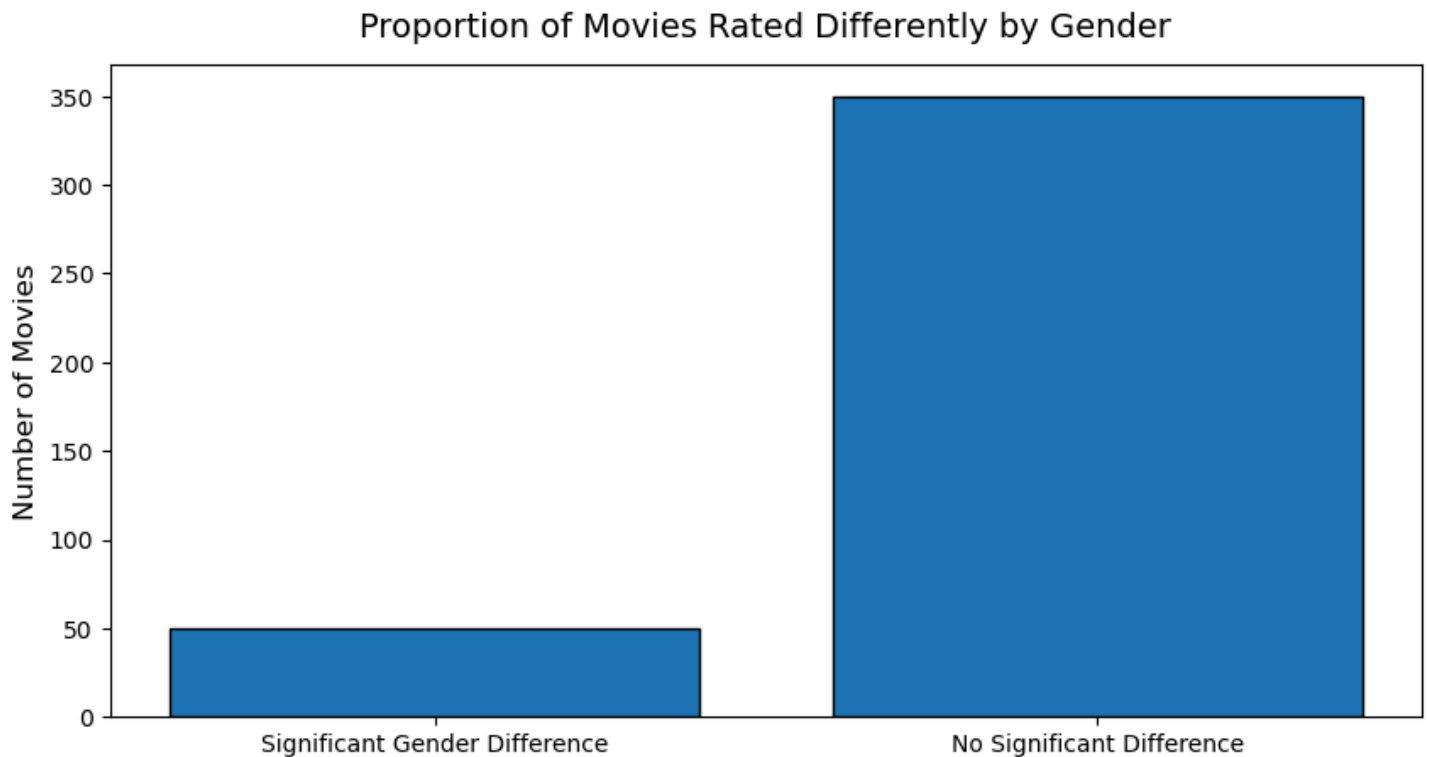
```
prop_diff = significant_count / tested_count
print(f"Movies tested: {tested_count}")
print(f"Movies with significant gender difference: {significant_count}")
print(f"Proportion: {prop_diff:.3f}")
```

```
Movies tested: 400
Movies with significant gender difference: 50
Proportion: 0.125
```

```
plt.figure(figsize=(10,5))

nonsig_count = tested_count - significant_count
labels = ['Significant Gender Difference', 'No Significant Difference']
values = [significant_count, nonsig_count]

bars = plt.bar(labels, values, edgecolor='black')
plt.title("Proportion of Movies Rated Differently by Gender", fontsize=14, pad=12)
plt.ylabel("Number of Movies", fontsize=12)
plt.show()
```



5. Do people who are only children enjoy 'The Lion King (1994)' more than people with siblings?

```
lion_king = df["The Lion King (1994)"]
only_child = df.iloc[:, 475].replace(-1, pd.NA)

# Drop missing
data = pd.DataFrame({"lion_king": lion_king, "only_child": only_child}).dropna()
```

```

# Split groups
only_chld = data[data["only_child"] == 1]["lion_king"]
siblings = data[data["only_child"] == 0]["lion_king"]

# Mann-Whitney U test
u_stat, p_val = mannwhitneyu(only_chld, siblings, alternative="two-sided")
r_rb = 1 - (2 * u_stat) / (len(only_chld) * len(siblings))

```

data

	lion_king	only_child
0	4.0	0
2	4.0	1
3	3.0	0
4	4.0	1
5	4.0	1
...	...	...
1090	3.5	0
1091	4.0	0
1092	3.5	0
1093	3.5	0

109
6 4.0 0

927 rows × 2 columns

```
mean_only_chld = only_chld.mean()
mean_siblings = siblings.mean()

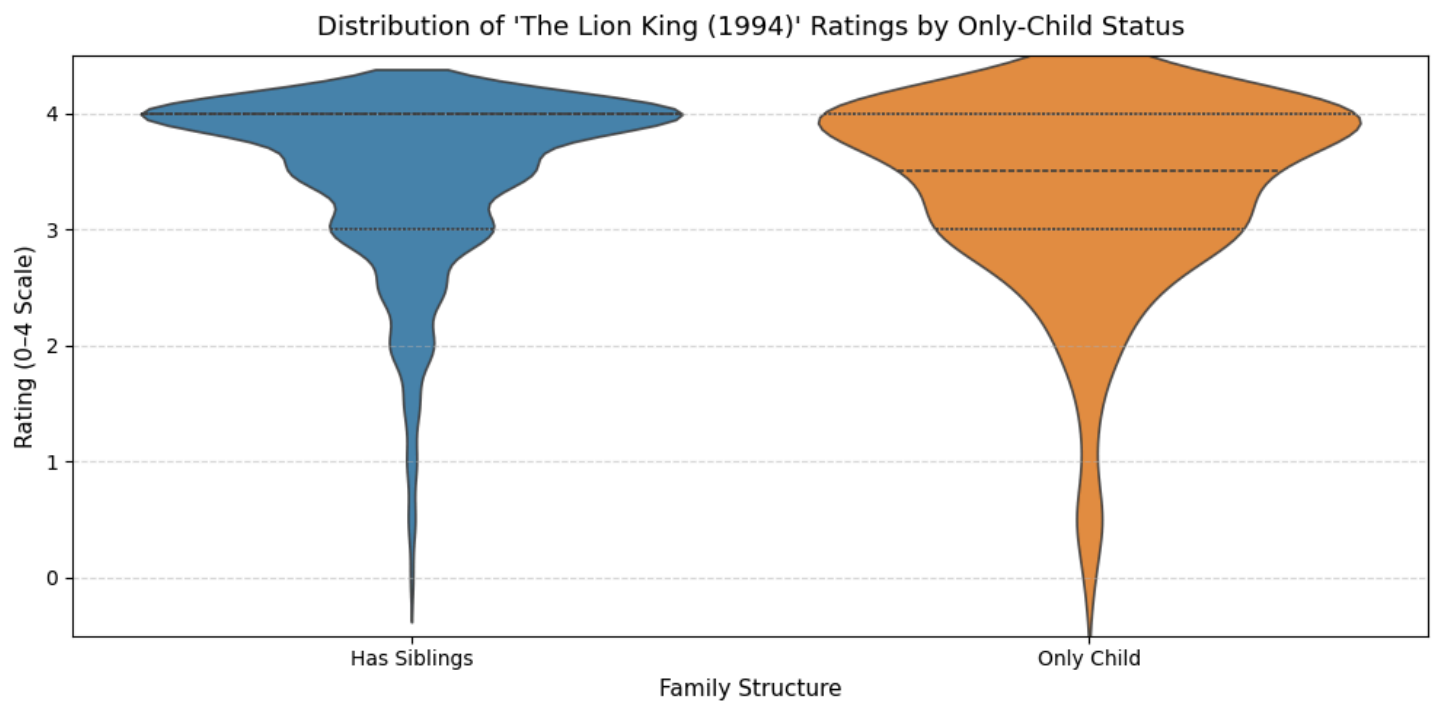
print(f"U = {u_stat:.1f}, p = {p_val:.3e}, r_rb = {r_rb:.3f}")
print(f"Only-child mean = {mean_only_chld:.2f}, With-siblings mean = {mean_siblings:.2f}")

U = 52929.0, p = 4.320e-02, r_rb = 0.097
Only-child mean = 3.35, With-siblings mean = 3.48

data["only_child_label"] = data["only_child"].replace({
    1: "Only Child",
    0: "Has Siblings"
})

plt.figure(figsize=(10,5))
sns.violinplot(
    data=data,
    x="only_child_label",
    y="lion_king",
    hue="only_child_label",
    inner="quartile",
    fill=True,
    legend=False,
    linewidth=1.2,
    alpha=0.9
)

plt.title("Distribution of 'The Lion King (1994)' Ratings by Only-Child Status",
    fontsize=13, pad=10)
plt.xlabel("Family Structure", fontsize=11)
plt.ylabel("Rating (0-4 Scale)", fontsize=11)
plt.ylim(-0.5, 4.5)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



6. What proportion of movies exhibit an “only child effect”, i.e. are rated different by viewers with siblings vs those without?

```

only_child = df.iloc[:, 475].replace(-1, pd.NA)

valid_cases = only_child.isin([0, 1])
only_child = only_child[valid_cases]
df_movies = df.loc[valid_cases, df.columns[0:400]]

alpha = 0.005
significant_count = 0
tested_count = 0

# Loop over movies
for col in df_movies.columns:
    ratings = df_movies[col]
    data = pd.DataFrame({"rating": ratings, "only_child": only_child}).dropna()

    if data["only_child"].nunique() == 2:
        only = data[data["only_child"] == 1]["rating"]
        sibs = data[data["only_child"] == 0]["rating"]

        if len(only) > 0 and len(sibs) > 0:
            u_stat, p_val = mannwhitneyu(only, sibs, alternative="two-sided")
            tested_count += 1
            if p_val < alpha:
                significant_count += 1

prop_diff = significant_count / tested_count
print(f"Movies tested: {tested_count}")
print(f"Movies with significant only-child effect: {significant_count}")

```

```
print(f"Proportion: {prop_diff:.3f}")
```

Movies tested: 400

Movies with significant only-child effect: 7

Proportion: 0.018

```
plt.figure(figsize=(10,5))
```

```
nonsig_count = tested_count - significant_count
```

```
labels = ['Significant only child effect', 'No Significant effect']
```

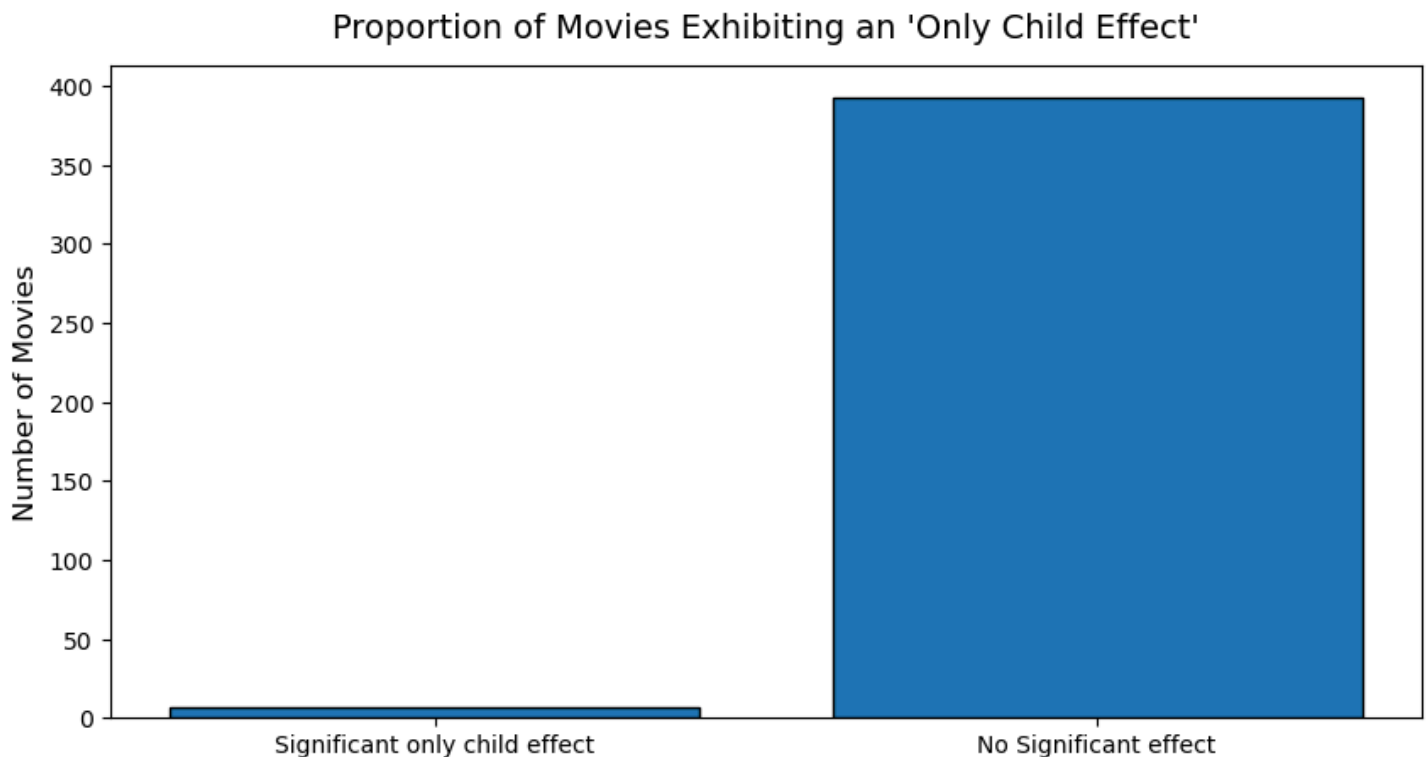
```
values = [significant_count, nonsig_count]
```

```
bars = plt.bar(labels, values, edgecolor='black')
```

```
plt.title("Proportion of Movies Exhibiting an 'Only Child Effect'", fontsize=14, pad=12)
```

```
plt.ylabel("Number of Movies", fontsize=12)
```

```
plt.show()
```



7. Do people who like to watch movies socially enjoy 'The Wolf of Wall Street (2013)' more than those who prefer to watch them alone?

```
wolf = df["The Wolf of Wall Street (2013)"]
```

```
social_pref = df.iloc[:, 476].replace(-1, pd.NA) # 0=social, 1=alone
```

```
data = pd.DataFrame({  
    "wolf": wolf,  
    "social_pref": social_pref  
}).dropna()
```

```

# Split groups
social = data[data["social_pref"] == 0]["wolf"]
alone = data[data["social_pref"] == 1]["wolf"]

# Mann-Whitney U test
u_stat, p_val = mannwhitneyu(social, alone, alternative="two-sided")
r_rb = 1 - (2 * u_stat) / (len(social) * len(alone))

mean_social = social.mean()
mean_alone = alone.mean()

print(f"U = {u_stat:.1f}, p = {p_val:.3e}, r_rb = {r_rb:.3f}")
print(f"Social watchers mean = {mean_social:.2f}, Solo watchers mean = {mean_alone:.2f}")

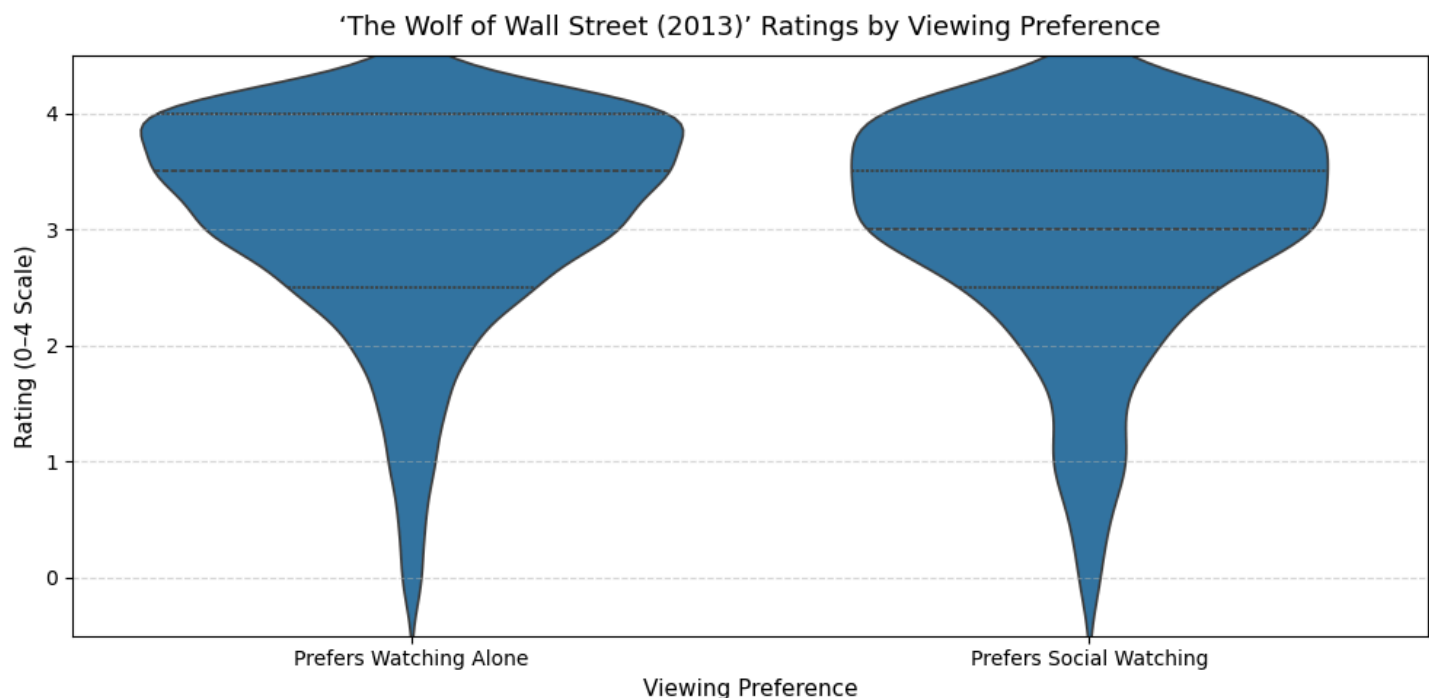
U = 49303.5, p = 1.128e-01, r_rb = 0.071
Social watchers mean = 3.03, Solo watchers mean = 3.14

data["preference_label"] = data["social_pref"].replace({
    0: "Prefers Social Watching",
    1: "Prefers Watching Alone"
})

plt.figure(figsize=(10,5))
sns.violinplot(
    data=data,
    x="preference_label",
    y="wolf",
    inner="quartile",
    linewidth=1.2
)

plt.title("`The Wolf of Wall Street (2013)` Ratings by Viewing Preference", fontsize=13,
pad=10)
plt.xlabel("Viewing Preference", fontsize=11)
plt.ylabel("Rating (0-4 Scale)", fontsize=11)
plt.ylim(-0.5, 4.5)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



8. What proportion of movies exhibit such a “social watching” effect?

```
movie_cols = df.columns[1:401]
social_col = df.columns[476]

# Removing invalid codes (-1 means missing/no response)
data = df[df[social_col] != -1]

sig_movies = 0
alpha = 0.005 # significance threshold
p_values = []

for movie in movie_cols:
    group_social = data[data[social_col] == 0][movie].dropna()
    group_solo = data[data[social_col] == 1][movie].dropna()

    # Skip movies with no data in one group
    if len(group_social) > 0 and len(group_solo) > 0:
        stat, p = mannwhitneyu(group_social, group_solo, alternative='two-sided',
nan_policy='omit')
        p_values.append(p)
        if p < alpha:
            sig_movies += 1

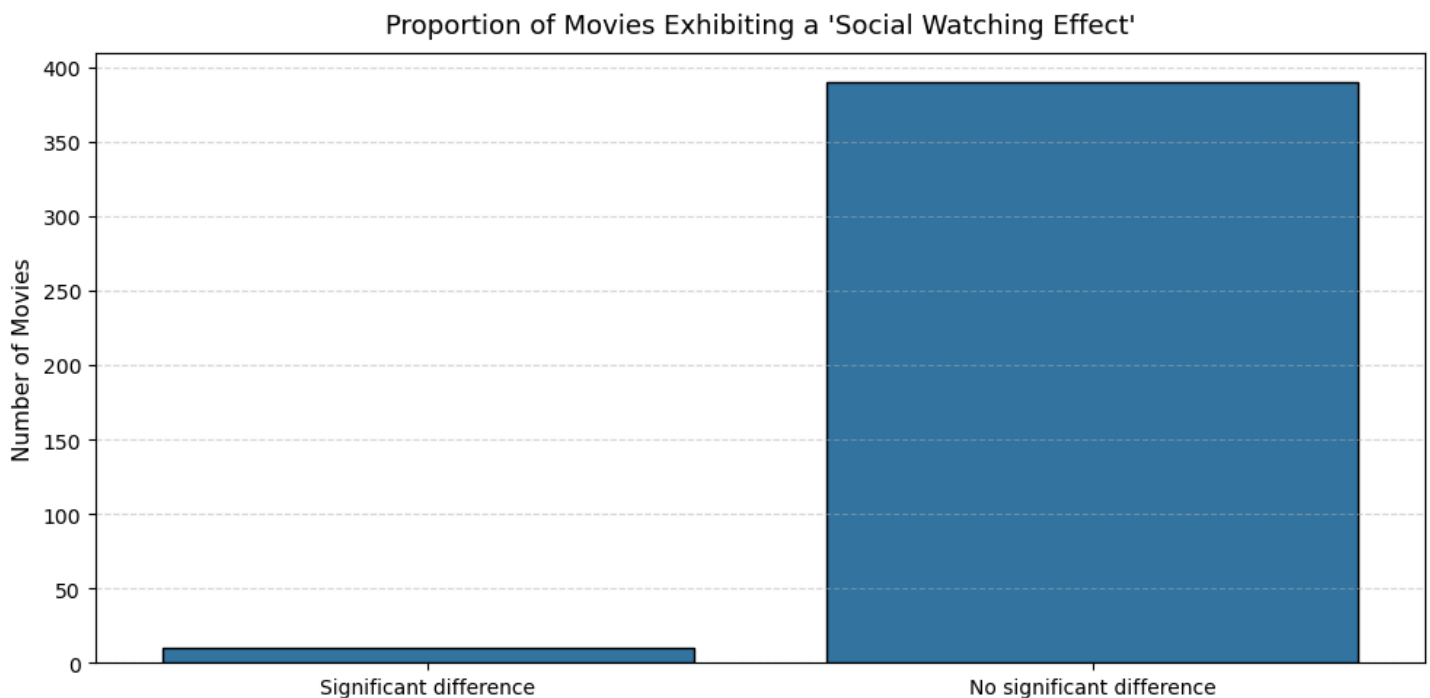
# --- Compute proportion ---
total_movies = len(movie_cols)
proportion = sig_movies / total_movies

print(f"Movies tested: {total_movies}")
print(f"Movies with significant social-watching effect: {sig_movies}")
print(f"Proportion: {proportion:.3f}")
```

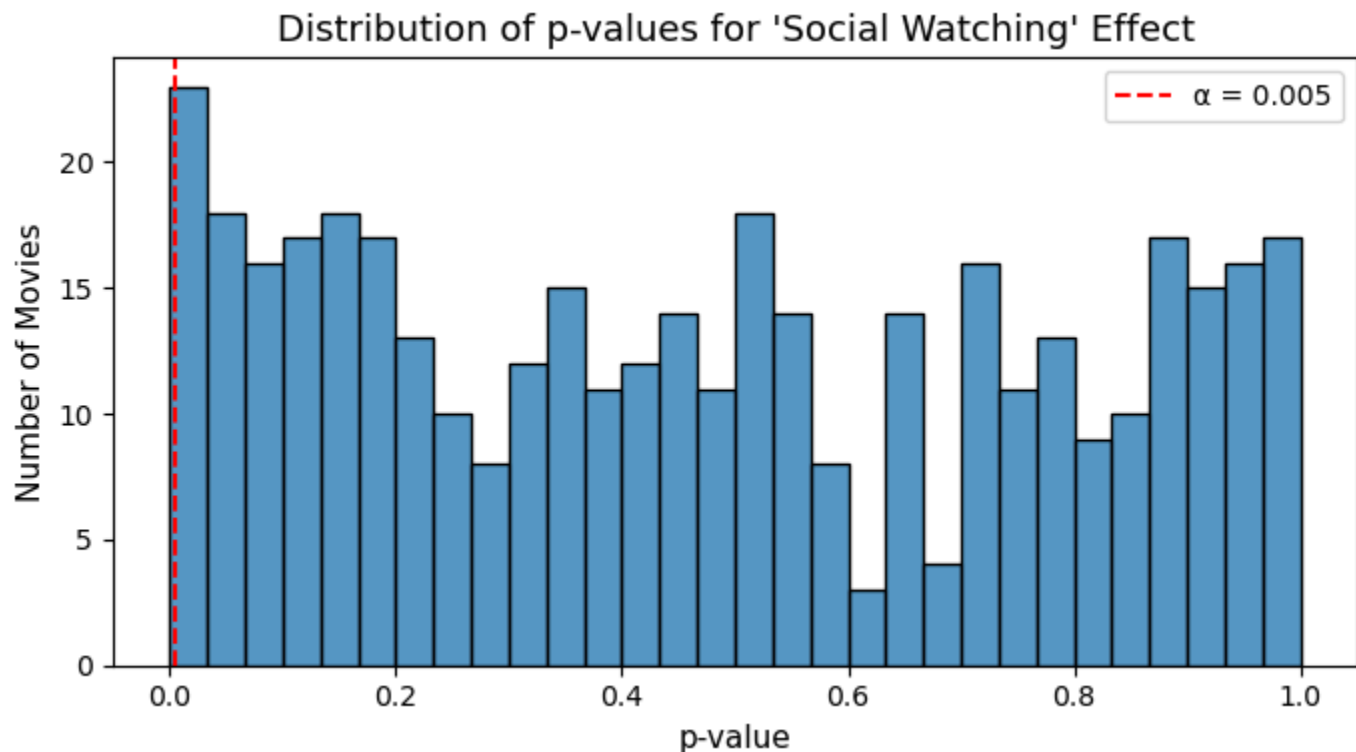
Movies tested: 400
Movies with significant social-watching effect: 10
Proportion: 0.025

```
results = {
    "Significant difference": sig_movies,
    "No significant difference": total_movies - sig_movies
}

plt.figure(figsize=(10,5))
sns.barplot(x=list(results.keys()), y=list(results.values()), edgecolor="black")
plt.title("Proportion of Movies Exhibiting a 'Social Watching Effect'", fontsize=13,
pad=10)
plt.ylabel("Number of Movies", fontsize=11)
plt.xlabel("")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(7,4))
sns.histplot(p_values, bins=30, edgecolor="black")
plt.axvline(alpha, color="red", linestyle="--", label=f" $\alpha = \{alpha\}$ ")
plt.title("Distribution of p-values for 'Social Watching' Effect", fontsize=13)
plt.xlabel("p-value", fontsize=11)
plt.ylabel("Number of Movies", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```



9. Is the ratings distribution of 'Home Alone (1990)' different than that of 'Finding Nemo (2003)'?

```
movie1 = 'Home Alone (1990)'
movie2 = 'Finding Nemo (2003)'

# Drop rows where either rating is missing
paired_data = df[[movie1, movie2]].dropna()

#Wilcoxon signed-rank test
stat, p = wilcoxon(paired_data[movie1], paired_data[movie2], alternative='two-sided')

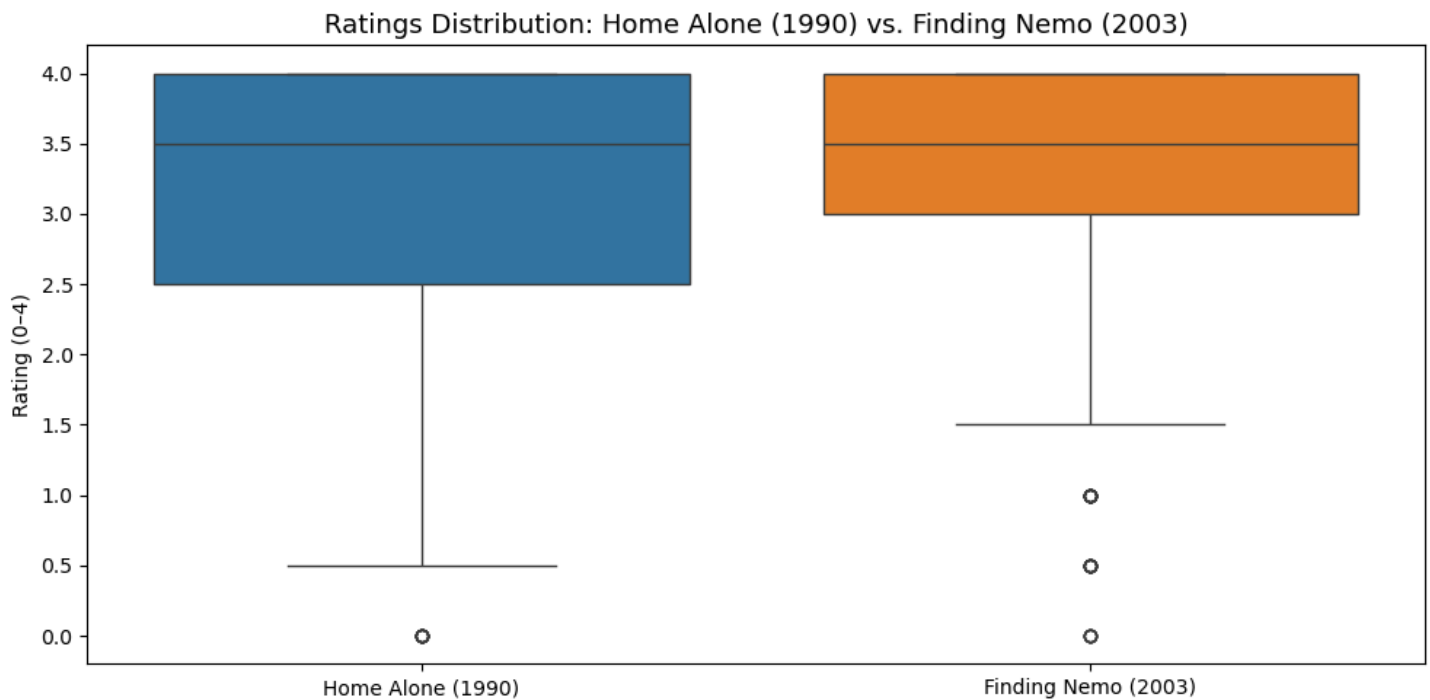
# Compute descriptive stats
mean1 = paired_data[movie1].mean()
mean2 = paired_data[movie2].mean()
n = len(paired_data)

print(f"n = {n}")
print(f"{movie1} mean = {mean1:.2f}")
print(f"{movie2} mean = {mean2:.2f}")
print(f"Wilcoxon statistic = {stat:.1f}, p = {p:.3e}")

n = 810
Home Alone (1990) mean = 3.13
Finding Nemo (2003) mean = 3.40
Wilcoxon statistic = 41298.5, p = 3.910e-17

plt.figure(figsize=(10,5))
sns.boxplot(data=paired_data[[movie1, movie2]], orient='v')
```

```
plt.title("Ratings Distribution: Home Alone (1990) vs. Finding Nemo (2003)",
         fontsize=13)
plt.ylabel("Rating (0-4)")
plt.tight_layout()
plt.show()
```



10. There are ratings on movies from several franchises ([‘Star Wars’, ‘Harry Potter’, ‘The Matrix’, ‘Indiana Jones’, ‘Jurassic Park’, ‘Pirates of the Caribbean’, ‘Toy Story’, ‘Batman’]) in this dataset. How many of these are of inconsistent quality, as experienced by viewers? [Hint: You can use the keywords in quotation marks featured in this question to identify the movies that are part of each franchise]

```
# Define franchise keywords
franchises = [
    'Star Wars', 'Harry Potter', 'The Matrix',
    'Indiana Jones', 'Jurassic Park',
    'Pirates of the Caribbean', 'Toy Story', 'Batman'
]

movie_cols = df.columns[1:401]

# Dictionary to store results
franchise_results = {}

alpha = 0.005

# Loop through each franchise
for franchise in franchises:
    # Find all movies containing this franchise keyword
    movies = [col for col in movie_cols if franchise.lower() in col.lower()]

    if len(movies) > 1: # need at least 2 movies to compare
```



```

# Prepare ratings for each movie
groups = [df[m].dropna() for m in movies]
# Kruskal-Wallis H-test
stat, p = kruskal(*groups)
franchise_results[franchise] = {
    'n_titles': len(movies),
    'H': stat,
    'p': p,
    'inconsistent': p < alpha
}

```

```

# Summary
inconsistent_count = sum(v['inconsistent'] for v in franchise_results.values())
total_franchises = len(franchise_results)

print("Franchise consistency results:")
for name, res in franchise_results.items():
    print(f"{name}: n = {res['n_titles']}, H = {res['H']:.2f}, p = {res['p']:.3e}, inconsistent = {res['inconsistent']}")

print(f"\nTotal franchises tested: {total_franchises}")
print(f"Franchises with inconsistent quality: {inconsistent_count}")

```

```

Franchise consistency results:
Star Wars: n = 6, H = 230.58, p = 8.016e-48, inconsistent = True
Harry Potter: n = 4, H = 3.33, p = 3.433e-01, inconsistent = False
The Matrix: n = 3, H = 48.38, p = 3.124e-11, inconsistent = True
Indiana Jones: n = 4, H = 45.79, p = 6.273e-10, inconsistent = True
Jurassic Park: n = 3, H = 46.59, p = 7.637e-11, inconsistent = True
Pirates of the Caribbean: n = 3, H = 20.64, p = 3.290e-05, inconsistent = True
Toy Story: n = 3, H = 24.39, p = 5.066e-06, inconsistent = True
Batman: n = 3, H = 190.53, p = 4.225e-42, inconsistent = True

```

```

Total franchises tested: 8
Franchises with inconsistent quality: 7

```

```

all_data = []

for franchise in franchises:
    movies = [col for col in movie_cols if franchise.lower() in col.lower()]
    if len(movies) > 1:
        melted = df[movies].melt(var_name='Movie', value_name='Rating')
        melted['Franchise'] = franchise
        all_data.append(melted)

plot_df = pd.concat(all_data, ignore_index=True)

plt.figure(figsize=(10,5))
sns.violinplot(data=plot_df, x='Franchise', y='Rating', inner='quartile', scale='width')
plt.title("Viewer Rating Distributions by Franchise")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()

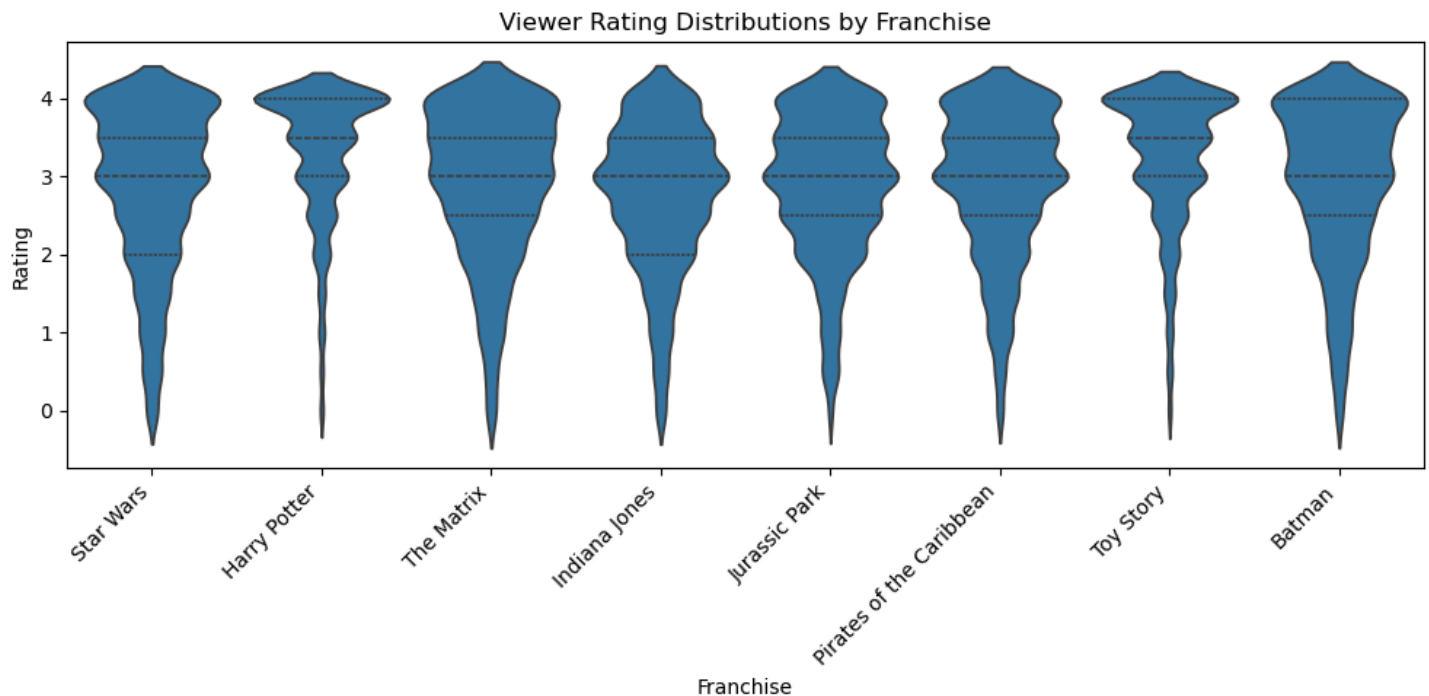
```

```
plt.show()
```

C:\Users\adars\AppData\Local\Temp\ipykernel_13112\426278216.py:13: FutureWarning:

The `scale` parameter has been renamed and will be removed in v0.15.0. Pass `density\_norm='width'` for the same effect.

```
sns.violinplot(data=plot_df, x='Franchise', y='Rating', inner='quartile',
scale='width')
```



Extra Credit: Tell us something interesting and true (supported by a significance test of some kind) about the movies in this dataset that is not already covered by the questions above [for 5% of the grade score].

We want to check if there is gender bias towards female led movies and we will be using permutation test here

```
female_led = [
    "Alien (1979)", "Aliens (1986)", "Black Swan (2010)", "Carrie (1976)",
    "Clueless (1995)", "Divine Secrets of the Ya-Ya Sisterhood (2002)",
    "Fatal Attraction (1987)", "Ghostbusters (2016)", "Girl, Interrupted (1999)",
    "My Best Friend's Wedding (1997)", "Rosemary's Baby (1968)", "Room (2015)",
    "Runaway Bride (1999)", "Terms of Endearment (1983)", "The Blue Lagoon (1980)",
    "The Descent (2005)", "The Others (2001)", "The Proposal (2009)",
    "The Silence of the Lambs (1991)", "Uptown Girls (2003)"
]

# --- Extract only movie columns ---
movie_cols = df.columns[1:401] # assuming first column = ID or demographics

# --- Find which columns match female-led titles ---
female_cols = [col for col in movie_cols if any(name in col for name in female_led)]
other_cols = [col for col in movie_cols if col not in female_cols]
```

```

df["female_led_mean"] = df[female_cols].mean(axis=1, skipna=True)
df["other_mean"] = df[other_cols].mean(axis=1, skipna=True)

valid = df[["female_led_mean", "other_mean"]].dropna()

obs_diff = valid["female_led_mean"].mean() - valid["other_mean"].mean()

res = permutation_test(
    (valid["female_led_mean"], valid["other_mean"]),
    statistic=lambda x, y: np.mean(x) - np.mean(y),
    permutation_type='independent',
    n_resamples=10000,
    alternative='two-sided',
    random_state=42
)

print(f"Observed mean difference: {obs_diff:.3f}")
print(f"p-value: {res.pvalue:.5f}")

Observed mean difference: 0.026
p-value: 0.32657

melted = valid.melt(value_vars=["female_led_mean", "other_mean"],
                    var_name="Group", value_name="Average Rating")

sns.violinplot(x="Group", y="Average Rating", data=melted, inner="box", cut=0)
plt.title("Permutation Test: Female-led vs Other Movies")
plt.show()

```

